MAE 162D/E Capstone Lab

Week 9 – Rover Motion Trajectory Generator

Lab Document

Lab Objectives

This lab follows Week 6 lab and implements the P position controller for translational and rotational motion.

By the end of this lab, students will be able to:

1. Understand the overall rover motion control architecture and the role of the rover dynamics generator within the closed-loop control system.
2. Design and implement a proportional (P) controller for regulating rover translational and angular motion using position and heading error.
3. Generate trajectory command sequences using accumulated distance (odometer) and global heading direction.
4. Simulate rover motion in Simulink and verify trajectory performance using XY plots.
5. Implement the motion command generator and P controller on Arduino hardware.
6. Validate controller performance by comparing simulation results with physical rover motion.
7. Modify command sequences to produce smooth turning trajectories instead of sharp 90° turns.

Overall Rover Driving Software Structure

Figure 1 shows the block diagram of the overall rover driving software structure. We finished the yellow and green blocks in week 6. This week, we will focus on the rover dynamics generator (red box).

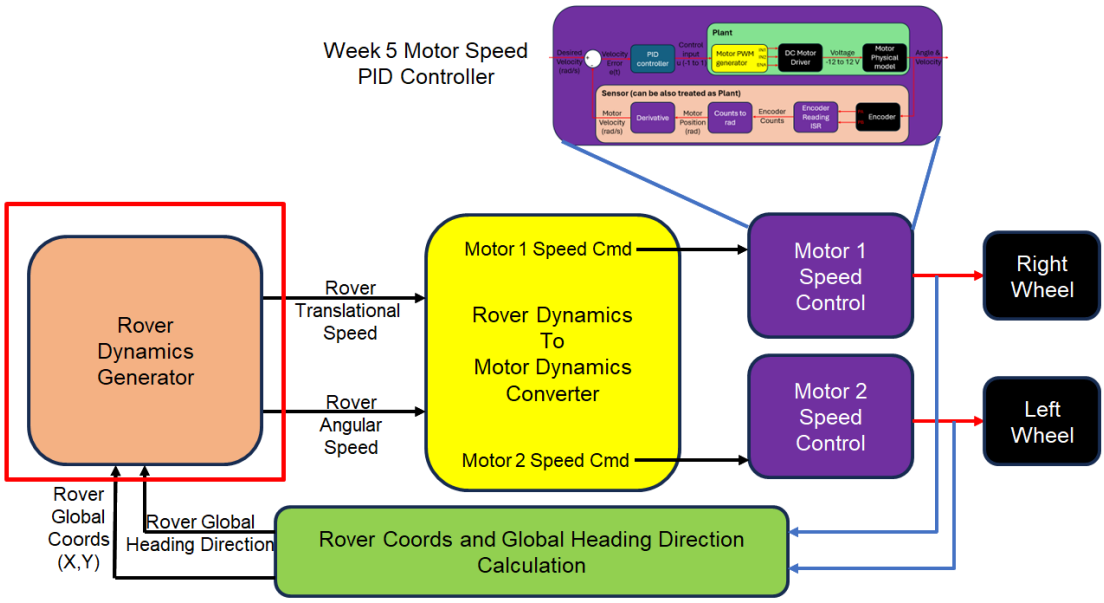

Figure 1, Rover motion block diagram

The rover dynamics generator is responsible for generating the rover translational and angular velocity command. In this lab, we will use a simple P controller based on the position and angular position error signal. While the rover angular position feedback is the rover global heading direction we calculated in week 6, the translational position feedback will be the accumulated distance that the rover has gone through, similar to the odometer in your car. This number will be set to zero at the rover power up and keep adding while the rover moves around. The block diagram can be shown in Figure 2, Note that this rover dynamics generator is the easy version for your reference and not the unique one. You can come up with more advanced command generators which take in coordinates x and y directly by yourself. Also, a more advanced command generator which allows the rover to travel in curve line will also be introduced later in this course.

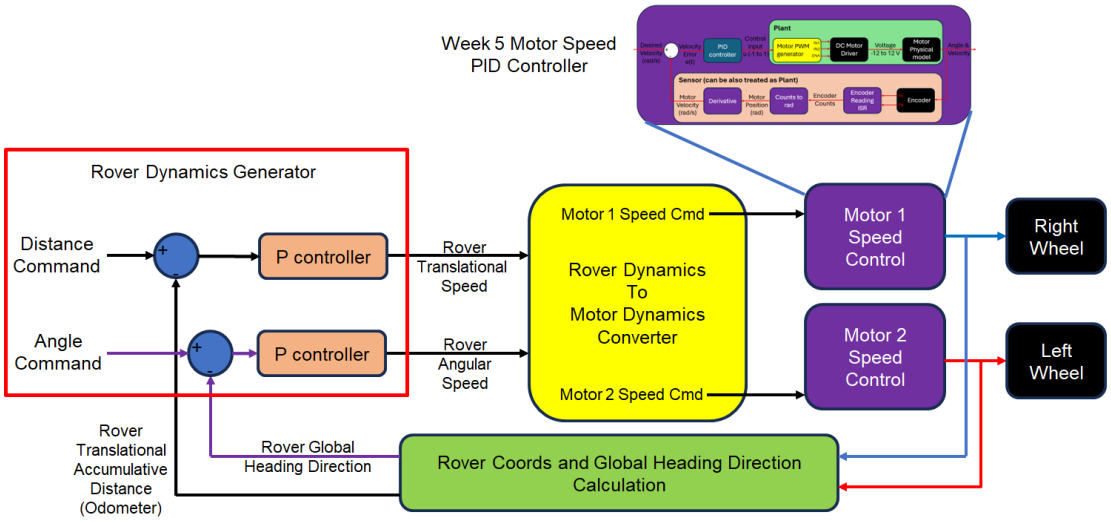

Figure 2, Rover dynamics generator block diagram

Simulink simulation of Rover Motion

As mentioned in the lecture, this block diagram can be implemented in Simulink for you to simulate the rover's motion without any hardware as shown in Figure 3 (Open and run DifferentialDrive_MotorControl.m). For our projects, although testing your rover is easy, the simulation model should be built for your practice. This will be very useful especially when hardware tests are very expensive.

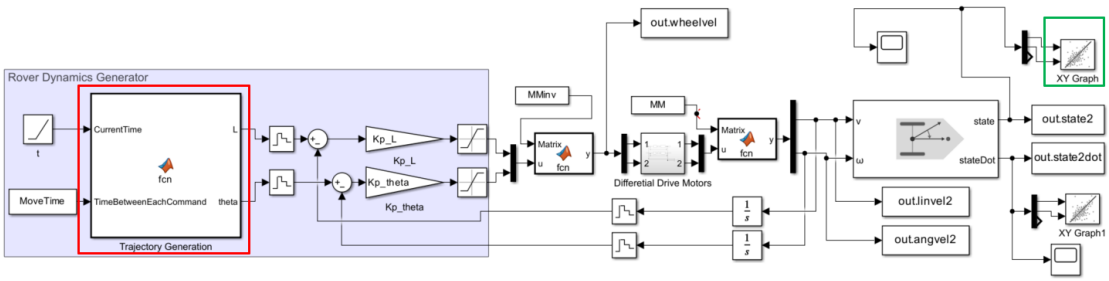

Figure 3, Rover motion simulation in Simulink

The Simulink block diagram is completed, and you will only need to modify the command sequence in the trajectory generation function (red box). After assigning the proper command sequence, run the simulation and check the rover trajectory in the XY graph (green box).

The command consists of two elements – the accumulated distance (odometer) and the global heading direction. To make a square motion, the commands can be shown in Figure 4.

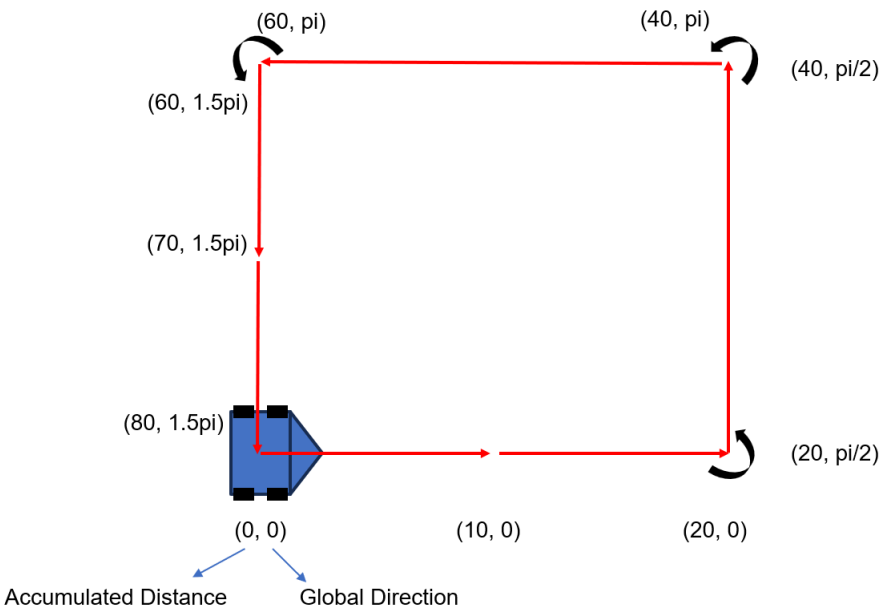

Figure 4, Accumulated distance and global direction commands illustration

You will need to assign this command array into the trajectory generation function (red box in Figure 3) as shown in Figure 5.

```
function [L, theta]= fcn(CurrentTime, TimeBetweenEachCommand)

L_array     = [0 10 20   20   40]; % Finish this distance commands array
Theta_array = [0  0  0 pi/2 pi/2]; % Finish this angle commands array


index = floor(CurrentTime/TimeBetweenEachCommand) + 1;
if index>length(L_array)
    index = length(L_array);
end
L = L_array(index);
theta = Theta_array(index);
```

Figure 5, Accumulative distance and global direction commands illustration

Task 1:

- Assign the completed commands array in Figure 5 and run the simulation.
- Screenshot the XY graph and see if the trajectory is the desired square.

Hardware Implementation of Command Generator

After the simulation is successful, we will implement this command generator into the actual hardware using the Arduino Mega microcontroller. This example is provided for your reference. You will simply need to finish the p controller for translational and angular position as shown in Figure 6.

```
float TranslationalSpeedGenerator(float CurrentPos, float DesiredPos)
{
  float kp = L_kp;
  // Hint: Implement the p controller from position error to generator speed cmd
  // Step 1: caculate the position error from DesiredPos and CurrentPos
  // float PosError = ??
  float Cmd;
  // Step 2: Calculate the command from PosError and kp
  //Cmd = ??
  // Step 3: If Cmd is grater than saturation, set it equal to saturation
  return Cmd;
}
float RotationalSpeedGenerator(float CurrentAngle, float DesiredAngle)
{
  float kp = theta_kp;
  // Hint: Implement the p controller from angle error to generator angular speed cmd
  // Step 1: caculate the angle error from DesiredAngle and CurrentAngle
  // float AngleError = ??
  float Cmd;
  // Step 2: Calculate the command from AngleError and kp
  //Cmd = ??
  // Step 3: If Cmd is grater than saturation, set it equal to saturation
  return Cmd;
}
```

Figure 6, P controller for translational and angular position example code

After finishing the p controller, you will need to copy and paste your commands array from simulation in Task 1 to the Arduino code shown in Figure 7. You can also change the CommandTimeInterval, L_kp and theta_kp variables if necessary.

```
float TranslationalPosArray[] = { 0, 10, 20,   20,   30 }; // Put your commands array here
float RotationalAngleArray[]  = { 0,  0,  0, pi/2, pi/2 }; // Put your commands array here
float CommandTimeInteval = 5; //sec
float L_kp = 1; float theta_kp = 1;
float TranslationalVelCmdSaturation = 8; // inch/sec
float RotationalVelCmdSaturation = 2; // rad/sec
```

Figure 7, Commands array code example

Task 2:

- Finish the code with the hints.
- Use the trajectory you used in the simulation, test and see your rover move along the square trajectory.

Task 3: (Optional)

- Instead of making 90 degrees sharp turn, change your commands array so the rover makes smooth turns as shown in Figure 8.
- Show this result both in simulation and actual rover motion.
-

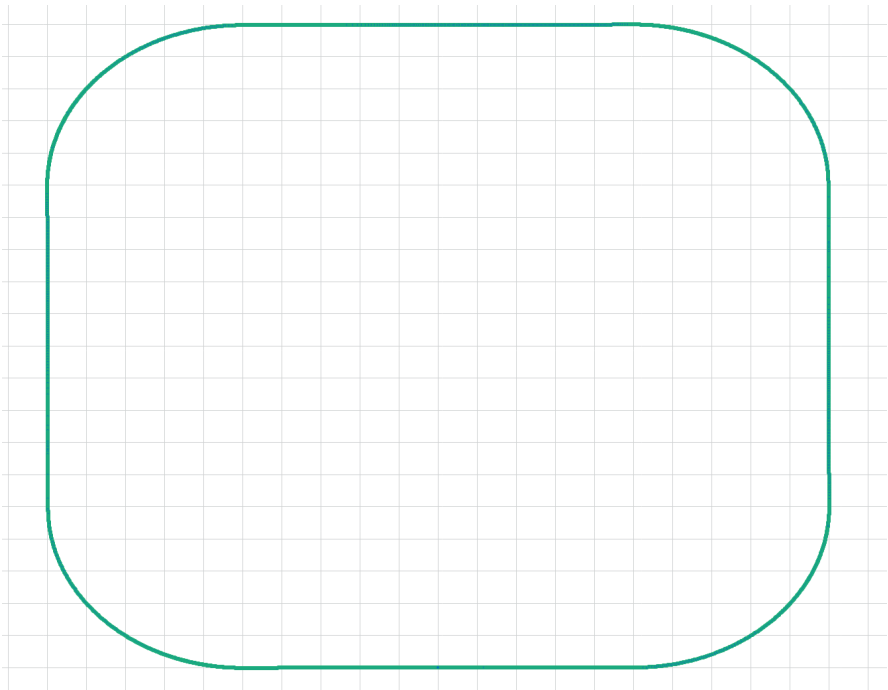

Figure 8, Smooth turn simulation

Deliverables

1. Finish Task 1, 2
2. Show TAs your screenshot in Task 1.
3. Show TAs your rover moving in square trajectory in Task 2.
4. Answer the following questions:
    i. Does your rover come back to its original position? If not, how much is off and why do you think this happened?
    ii. Do you think the position p controller is enough? Why?
    iii. Why do you think we need to have a saturation threshold for velocity and angular velocity commands?